```
#Practical No 7
from collections import deque

# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

vertices = list(graph.keys())

# Adjacency Matrix
matrix = [
    [1 if v in graph[u] else 0 for v in vertices]
    for u in vertices
]

print("Adjacency Matrix:")
print("  ", vertices)

for i in range(len(vertices)):
    print(vertices[i], matrix[i])

# Adjacency List
print("\nAdjacency List:")
for vertex in graph:
    print(vertex, ":", graph[vertex])

# Breadth First Search (BFS)
def bfs(start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    print("BFS Traversal:", end=" ")

    while queue:
        vertex = queue.popleft()
        print(vertex, end=" ")
```

```
        for neighbour in graph[vertex]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

    print()

# Depth First Search (DFS)
def dfs(start, visited=None):
    if visited is None:
        visited = set()

    visited.add(start)
    print(start, end=" ")

    for neighbour in graph[start]:
        if neighbour not in visited:
            dfs(neighbour, visited)

print("\nStarting from vertex A:")
bfs('A')

print("DFS Traversal:", end=" ")
dfs('A')
print()
```

#Sample Output :

```
Adjacency Matrix:
   ['A', 'B', 'C', 'D', 'E', 'F']
A [0, 1, 1, 0, 0, 0]
B [1, 0, 0, 1, 1, 0]
C [1, 0, 0, 0, 0, 1]
D [0, 1, 0, 0, 0, 0]
E [0, 1, 0, 0, 0, 1]
F [0, 0, 1, 0, 1, 0]

Adjacency List:
A : ['B', 'C']
B : ['A', 'D', 'E']
C : ['A', 'F']
D : ['B']
E : ['B', 'F']
F : ['C', 'E']
```

Starting from vertex A:
BFS Traversal: A B C D E F
DFS Traversal: A B D E F C